

Programowanie obiektowe

Wykład 11

Tworzenie aplikacji z graficznym interfejsem użytkownika w Windows Forms

Windows Forms – podstawy

- Biblioteka GUI w .NET do tworzenia aplikacji desktopowych dla Windows
- Opiera się na **Windows API (Win32)** (natywna wydajność i wygląd)
- Obsługuje okna, przyciski, formularze, listy, tabele itd.
- Działa w oparciu o **zdarzenia** (kliknięcia, zmiany itp.)

Cechy

- Tworzenie GUI wizualnie (designer) lub w kodzie
- Tylko na Windows
- Łatwa integracja z innymi bibliotekami .NET
- Świetna do nauki podstaw GUI i programowania zdarzeniowego
- Zastąpiona przez nowsze technologie (WPF, UWP, MAUI), ale wciąż aktywnie wspierana i używana w projektach biznesowych

Zalety

- Prosta i szybka w nauce
- Stabilna, dobrze udokumentowana
- Nadal wspierana

Wady

- Ograniczona responsywność i słabe wsparcie dla skalowania

Architektura aplikacji Windows Forms

- **Form** – główne okno aplikacji (`Form1` , `MainForm`)
- **Control** – wszystkie przyciski, pola, etykiety itp. dziedziczą z `System.Windows.Forms.Control`
- **Application.Run(form)** – uruchamia główną pętlę komunikatów Windows

Cykl życia aplikacji

1. `Main()` – punkt wejścia
2. `Application.Run(new Form1())` – start pętli zdarzeń
3. Użytkownik wchodzi w interakcję → zdarzenia są wywoływane

Struktura typowej aplikacji

```
Main()  
├── Application.Run(new Form1())  
│   ├── Form1_Load()  
│   ├── Kontrolki: Button, TextBox, etc.  
│   └── Obsługa zdarzeń: Click, KeyDown, itp.
```

Model zdarzeniowy (*event-driven programming*)

- Aplikacja reaguje na **zdarzenia**, a nie wykonuje kod liniowo
- Przykłady zdarzeń:
 - `Click` – kliknięcie przycisku
 - `TextChanged` – zmiana tekstu w `TextBox`
 - `SelectedIndexChanged` – zmiana zaznaczenia w `ListBox`
- Zdarzenia są obsługiwane przez **delegaty** (metody przypięte do zdarzenia)

```
button1.Click += Button1_Click;
```

```
private void Button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("Kliknięto przycisk!");
}
```

Kontrolki

Kontrolki to elementu interfejsu użytkownika (UI) – np. przycisk, pole tekstowe, lista

- Wszystkie dziedziczą z klasy bazowej `System.Windows.Forms.Control`
- Podstawowe kontrolki formularzy:
 - `Label` – Wyświetlanie tekstu (statycznego)
 - `TextBox` – Wprowadzanie tekstu
 - `Button` – Klikalny przycisk
 - `CheckBox` – Zaznaczalna opcja
 - `RadioButton` – Wybór jednej opcji z grupy
 - `ComboBox` – Lista rozwijana
 - `ListBox` – Lista pozycji z możliwością zaznaczania
 - `DataGridView` – Tabela danych z możliwością edycji
 - `DateTimePicker` – Wybór daty

Właściwości i zarzenia kontroltek

- Kluczowe właściwości kontroltek
 - `Text` – Treść wyświetlana (lub wpisywana)
 - `Name` – Nazwa kontrolki w kodzie
 - `Size` / `Location` – Rozmiar i pozycja na formularzu
 - `Dock` , `Anchor` , `Padding` , `Margin` – właściwości odpowiedzialne za rozmieszczenie
 - `Enabled` – Czy kontrolka jest aktywna
 - `Visible` – Czy jest widoczna
 - `BackColor` , `ForeColor` , `Font` – Styl wizualny
- Zdarzenia (events)
 - `Click` (np. `Button`)
 - `TextChanged` (np. `TextBox`)
 - `SelectedIndexChanged` (np. `ComboBox` , `ListBox`)

Tworzenie okna w kodzie – krok po kroku

- Okno możemy utworzyć ręcznie, np. w `Main()` :

```
// W pliku 'Program.cs'

namespace WinFormsApp
{
    internal static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            ApplicationConfiguration.Initialize();

            ...
        }
    }
}
```

Tworzenie okna w kodzie – krok po kroku

- Najpierw tworzymy nowe okno (`Form`)

```
Form form = new Form
{
    Text = "Witaj w Windows Forms",
    Width = 600,
    Height = 500,
    Font = new Font( "Segoe UI", 22 )
};
```

- Możemy od razu ustawić jego właściwości (np. `Font`).
- W przypadku ustawień czcionek, część kontroltek (np. `Label` , `TextBox` , `Button`) automatycznie odziedziczy to ustawienie.

Tworzenie okna w kodzie – krok po kroku

- Tworzymy etykietę tekstową (`Label`)

```
Label label = new Label  
{  
    Left = 20,  
    Top = 20,  
    AutoSize = true,  
    Text = "Hello, World!"  
};
```

- Dla niektórych kontroltek (np. `Label` , `Button` , `CheckBox`) można ustawić `AutoSize` .
- To sprawi, że szerokość i wysokość kontrolki dopasuje się do zawartości tekstowej i fonta.
- Następnie dodajemy kontrolkę do formularza.

```
form.Controls.Add(label);
```

- I uruchamiamy aplikację.

```
Application.Run(form);
```

Aplikacja – prosty kalkulator

- Kalkulator potrzebuje kontroltek tekstowych, które pozwolą na wprowadzenie tekstu (`TextBox`)

```
TextBox textBox1 = new TextBox
{
    Left = 20,
    Top = 20,
    Width = 260
};
form.Controls.Add(textBox1);

TextBox textBox2 = new TextBox
{
    Left = 300,
    Top = 20,
    Width = 260
};
form.Controls.Add(textBox2);
```

Aplikacja – prosty kalkulator

- Obliczenia będą wykonane po naciśnięciu przycisku (Button)

```
Button button = new Button
{
    Text = "Oblicz sumę",
    Left = 20,
    Top = 100,
    AutoSize = true
};
form.Controls.Add(button);
```

- Za wyświetlenie wyniku będzie odpowiadać etykieta

```
Label label = new Label
{
    Left = 20,
    Top = 190,
    AutoSize = true,
    Text = "Wynik: "
};
form.Controls.Add(label);
```

Aplikacja – prosty kalkulator

- Do przycisku dodajemy obsługę zdarzenia:

```
button.Click += (sender, e) =>
{
    // próbujemy przeparsować obie wartości
    if (double.TryParse(textBox1.Text, out double num1) &&
        double.TryParse(textBox2.Text, out double num2))
    {
        // obliczamy sumę
        double suma = num1 + num2;
        // wpisujemy wynik do kontrolki
        label.Text = $"Wynik: {suma}";
    }
    else
    {
        // Komunikat w razie niepowodzenia
        label.Text = "Wprowadź poprawne liczby.";
    }
};
```

- Składowa `Click` jest typu **delegat**:

```
public delegate void EventHandler(object? sender, EventArgs e);
```

- Zamiast **funkcji lambda**, moglibyśmy użyć wskazania na metodę klasy `Program`.

Ułożenie kontroltek

- Mechanizm układów Windows Forms różni się od tego ze Swing. Nie ma tu dokładnego odpowiednika *layout managerów* (takich jak np. `BorderLayout` , `GridLayout` , `BoxLayout`), ale istnieją pewne alternatywy, np. `FlowLayoutPanel`

```
FlowLayoutPanel flow = new FlowLayoutPanel
{
    Dock = DockStyle.Fill,
    FlowDirection = FlowDirection.LeftToRight,
    WrapContents = true,
    AutoSize = true
};
flow.Controls.Add(new Button { Text = "Przycisk 1" });
flow.Controls.Add(new Button { Text = "Przycisk 2" });
```

- Albo `TableLayoutPanel`

```
TableLayoutPanel table = new TableLayoutPanel
{
    RowCount = 2,
    ColumnCount = 2,
    Dock = DockStyle.Fill
};
table.Controls.Add(new Label { Text = "Imię" }, 0, 0);
table.Controls.Add(new TextBox(), 1, 0);
table.Controls.Add(new Label { Text = "Nazwisko" }, 0, 1);
table.Controls.Add(new TextBox(), 1, 1);
```

Ułożenie kontroltek

- Możemy też ręcznie pozycjonować kontrolki, a ich właściwości (`Dock` , `Anchor`) pozwalają ustawić, jak kontrolka ma się zachowywać przy zmianie rozmiaru okna:

```
textBox1.Width = form.ClientSize.Width - 40;  
textBox1.Anchor = AnchorStyles.Left | AnchorStyles.Right;  
  
textBox2.Top = 20 + textBox1.Bounds.Bottom;  
textBox2.Width = form.ClientSize.Width - 40;  
textBox2.Anchor = AnchorStyles.Left | AnchorStyles.Right;  
  
button.Top = 20 + textBox2.Bounds.Bottom;  
button.Anchor = AnchorStyles.Left;  
  
label.Top = 20 + button.Bounds.Bottom;  
label.Anchor = AnchorStyles.Left;
```

- W praktyce kontrolki w oknie układa się korzystając z narzędzi wizualnych.
- Gdy tworzymy formularz (np. `Form1`), powstają trzy pliki powiązane z jednym oknem:
 - `Form1.cs` - główna logika aplikacji (tzw. *code-behind*)
 - `Form1.Designer.cs` - kod generowany automatycznie przez projektanta formularzy
 - `Form1.resx` - plik zasobów (np. teksty lokalizowane, obrazy, ikonki)

Edytor osób (Person)

Aplikacja – edytor osób

- Zaczniemy od stworzenia klasy reprezentującej pojedynczą osobę (najlepiej w pliku `Person.cs`)

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public override string ToString()
    {
        return $"{Name} ({Age})";
    }
}
```

- Aby móc osoby prawidłowo wyświetlać w kontrolce listy (`ListBox`), klasa `Person` powinna zawierać metodę `ToString()`
- Interfejs (formularz `PersonEditor`) możemy zaprojektować w edytorze wizualnym:
 - Dodajemy kontrolkę `ListBox` do formularza
 - Nadajemy jej nazwę, np. `listBox`
 - Opcjonalnie: możemy ustawić `Dock = Fill` lub odpowiedni rozmiar

Aplikacja – edytor osób

- W *code-behind* możemy stworzyć listę osób (jako prywatne pole w klasie `PersonEditor`)

```
private List<Person> people = new List<Person>
{
    new Person { Name = "Anna", Age = 30 },
    new Person { Name = "Tomek", Age = 25 },
    new Person { Name = "Basia", Age = 40 }
};
```

- Kontrolkę wypełnimy w momencie załadowania okienka (zdarzenie `Load` formularza)
 - W edytorze klikamy formularz
 - Wybieramy zakładkę zdarzeń (⚡)
 - Dwukrotnie klikamy `Load`

```
private void PersonEditor_Load(object sender, EventArgs e)
{
    foreach (var person in people)
    {
        listBox.Items.Add(person);
    }
}
```

- Lista osób powinna być widoczna w oknie.

Dodawanie osób

- Krok pierwszy to dodanie (w projektancie) przycisku (albo paska narzędzi `ToolStrip` lub menu `MenuStrip`)
- Do przycisku dołączamy obsługę zdarzenia `Click`
- Nową osobę możemy dodać bezpośrednio do kontrolki:

```
listBox.Items.Add(new Person() { Name = "Piotr", Age = 20 });
```

- Lepszym rozwiązaniem jest traktowanie listy `people` jako naszego „źródła prawdy” (*Single Source of Truth*) – to ją będziemy edytować, a na tej podstawie aktualizować widok:

```
people.Add(new Person() { Name = "Piotr", Age = 20 });  
RefreshList();
```

- Gdzie:

```
private void RefreshList()  
{  
    listBox.Items.Clear();  
    foreach (var person in people)  
    {  
        listBox.Items.Add(person);  
    }  
}
```

Wprowadzanie danych osoby

- Przed dodaniem osoby, będziemy otwierać okno dialogowe z prośbą o podanie danych.
- W tym celu (w projektancie) tworzymy nowy formularz `PersonForm` :
 - Dodajemy `TextBox` o nazwie `txtName` (etykieta: Imię)
 - Dodajemy `TextBox` o nazwie `txtAge` (etykieta: Wiek)
 - Dodajemy dwa przyciski: `btnOK` , `btnCancel` z odpowiednimi właściwościami:
 - `btnOK.DialogResult = DialogResult.OK`
 - `btnCancel.DialogResult = DialogResult.Cancel`
 - Ustawiamy `AcceptButton = btnOK` , `CancelButton = btnCancel` (we właściwościach formularza)
- W pliku `PersonForm.cs`

```
public partial class PersonForm : Form
{
    public Person Result { get; private set; }

    public PersonForm()
    {
        InitializeComponent();
    }

    private void btnOK_Click(object sender, EventArgs e) { /* ... */ }
}
```

Formularz dodawania osoby

- Obsługujemy zdarzenie kliknięcia Ok:

```
private void btnOK_Click(object sender, EventArgs e)
{
    string name = txtName.Text.Trim();
    if (!int.TryParse(txtAge.Text.Trim(), out int age))
    {
        MessageBox.Show("Wiek musi być liczbą całkowitą.");
        this.DialogResult = DialogResult.None; // Zablokuj zamknięcie
        return;
    }

    if (string.IsNullOrEmpty(name))
    {
        MessageBox.Show("Imię nie może być puste.");
        this.DialogResult = DialogResult.None;
        return;
    }

    Result = new Person { Name = name, Age = age };
    // zamknięcie formularza nastąpi automatycznie dzięki DialogResult.OK
}
```

- Odczytujemy dane z kontrolki i wpisujemy do publicznie dostępnego pola `Result`.

Użycie okna dialogowego do dodawania osoby

- W głównym formularzu:

```
private void btnAddPerson_Click(object sender, EventArgs e)
{
    using (var dialog = new PersonForm())
    {
        if (dialog.ShowDialog() == DialogResult.OK)
        {
            Person newPerson = dialog.Result;
            people.Add(newPerson);
            RefreshList();
        }
    }
}
```

- Tworzymy okno dialogowe.
- Otwieramy je przy pomocy `ShowDialog()`.
- Po zamknięciu, sprawdzamy wynik (jakim przyciskiem zostało zamknięte, np. `DialogResult.OK`).
- Odczytujemy utworzoną osobę (`Result`).
- Dodajemy do listy `people` (SSOT).
- Odświeżamy widok (zawartość kontroli `listBox`).

Edycja danych osoby

- Wymaga to pewnej modyfikacji formularza `PersonForm`
- Dodajemy przeciążony konstruktor:

```
public PersonForm(Person personToEdit) : this()
{
    // Wstępne uzupełnienie pól formularza
    txtName.Text = personToEdit.Name;
    txtAge.Text = personToEdit.Age.ToString();
}
```

- **Uwaga:** pamiętajmy o wywołaniu konstruktora bezparametrowego (`: this()`), który inicjalizuje komponenty.

Edycja danych osoby

- W głównym formularzu dodajemy przycisk „Edytuj” i jego obsługę

```
private void btnEditPerson_Click(object sender, EventArgs e)
{
    // Sprawdźmy, czy cokolwiek zaznaczono
    if (listBox.SelectedItem is not Person selectedPerson)
    {
        MessageBox.Show("Wybierz osobę do edycji.");
        return;
    }

    // Przekazanie osoby do formularza edycji
    using (var dialog = new PersonForm(selectedPerson))
    {
        if (dialog.ShowDialog() == DialogResult.OK)
        {
            // Nadpisujemy dane obiektu referencyjnie
            selectedPerson.Name = dialog.Result.Name;
            selectedPerson.Age = dialog.Result.Age;

            // Odświeżamy widok
            RefreshList();
        }
    }
}
```

Edycja danych osoby

- Odświeżenie widoku (wywołanie `RefreshList()`) było konieczne, ponieważ `ListBox` nie odświeża się automatycznie nawet po zmianie danych w obiekcie.
- Najprostsze rozwiązanie to wyczyszczenie i ponowne dodanie elementów do `ListBox`
- Możemy też sztucznie wymusić odświeżenie konkretnego elementu:

```
int index = listBox.SelectedIndex;  
listBox.Items[index] = listBox.Items[index];
```

- Alternatywą jest użycie `BindingList<T>` zamiast `List<T>`.

Usuwanie osób

- Usunięcie osoby jest proste, ale przedtem warto poprosić użytkownika o potwierdzenie:

```
private void btnDelete_Click(object sender, EventArgs e)
{
    if (listBox.SelectedItem is not Person selectedPerson)
    {
        MessageBox.Show("Najpierw wybierz osobę do usunięcia.");
        return;
    }

    // Potwierdzenie od użytkownika
    var result = MessageBox.Show(
        $"Czy na pewno chcesz usunąć {selectedPerson.Name} ({selectedPerson.Age} lat)?",
        "Potwierdzenie usunięcia",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        people.Remove(selectedPerson);
        RefreshList();
    }
}
```

Edytor osób – opcja zapisu

- Okno z pytaniem o nazwę pliku (`SaveFileDialog`) możemy przeciągnąć do formularza w projektancie lub utworzyć je w kodzie

```
using System.Text.Json;
using System.IO;

private void btnSave_Click(object sender, EventArgs e)
{
    using (SaveFileDialog saveDialog = new SaveFileDialog())
    {
        saveDialog.Filter = "Pliki JSON (*.json)|*.json|Wszystkie pliki (*.*)|*.*";
        saveDialog.Title = "Zapisz listę osób";

        if (saveDialog.ShowDialog() == DialogResult.OK)
        {
            string path = saveDialog.FileName;

            // ... zapisujemy dane do pliku
        }
    }
}
```

- `SaveFileDialog` służy jedynie do zapytania o nazwę pliku, nie zajmuje się zapisem
- `Filter` pozwala skonfigurować filtry do wybrania odpowiednich rozszerzeń wyświetlanych plików

Zapis osób do pliku JSON

- Serializacja
- Uwaga na wymagane przestrzenie nazw: `System.Text.Json` i `System.IO`.

```
try
{
    string json = JsonSerializer.Serialize(people,
        new JsonSerializerOptions { WriteIndented = true });
    File.WriteAllText(path, json);

    MessageBox.Show("Zapisano pomyślnie.", "Sukces",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
}
catch (Exception ex)
{
    MessageBox.Show($"Błąd podczas zapisu:\n{ex.Message}", "Błąd",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

Edytor osób – opcja odczytu

- Okno z pytaniem nazwę pliku do odczytu (`OpenFileDialog`) działa bardzo podobnie do `SaveFileDialog`

```
using System.Text.Json;
using System.IO;

private void btnLoad_Click(object sender, EventArgs e)
{
    using (OpenFileDialog openFileDialog = new OpenFileDialog())
    {
        openFileDialog.Filter = "Pliki JSON (*.json)|*.json|Wszystkie pliki (*.*)|*.*";
        openFileDialog.Title = "Wczytaj listę osób";

        if (openFileDialog.ShowDialog() == DialogResult.OK)
        {
            string path = openFileDialog.FileName;

            /* ... */
        }
    }
}
```

Odczyt danych z pliku JSON

■ Deserializacja

```
try
{
    string json = File.ReadAllText(path);
    var loadedPeople = JsonSerializer.Deserialize<List<Person>>(json);

    if (loadedPeople != null)
    {
        people = loadedPeople;
        RefreshList();
        MessageBox.Show("Wczytano pomyślnie.", "Sukces",
                        MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
    else
    {
        MessageBox.Show("Nie udało się odczytać danych z pliku.", "Błąd",
                        MessageBoxButtons.OK, MessageBoxIcon.Warning);
    }
}
catch (Exception ex)
{
    MessageBox.Show($"Błąd podczas odczytu:\n{ex.Message}", "Błąd",
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
}
```

Rozwiązanie alternatywne – kontrolka DataGridView

- Kontrolka `DataGridView` pozwoli na wyświetlanie danych w formacie tabeli
- Pozwala na automatyczne tworzenie kolumn na podstawie podpiętych danych (`AutoGenerateColumns = true`)
- Umożliwia edycję bezpośrednio w tabeli, w tym dodawanie i usuwanie wierszy (`AllowUserToAddRows = true` , `AllowUserToDeleteRows = true`)
- Do automatycznej aktualizacji `DataGridView` i synchronizacji zmian z listą `people` niezbędne jest użycie `BindinList<T>` :

```
private BindingList<Person> people = new BindingList<Person>();
```

- Teraz wystarczy wskazać listę `people` jako dane kontrolki:

```
dataGridView1.DataSource = people;
```

Grafika i animacja

Grafika i animacja

Animacja odbijającej się piłki

- W formularzu okna potrzebujemy danych piłki (położenie, wektor prędkości, rozmiar) oraz `Timera` do obsługi animacji (wywoływania cyklicznych zdarzeń):

```
int ballX = 150;  
int ballY = 50;  
int ballSize = 50;  
  
int dx = 4; // prędkość w poziomie  
int dy = 3; // prędkość w pionie  
  
Timer timer;
```

- `Timer` pochodzi z przestrzeni nazw `System.Windows.Forms`

Grafika i animacja

- Za rysowanie piłki będzie odpowiadać metoda `OnPaint()`

```
protected override void OnPaint(PaintEventArgs e)
{
    base.OnPaint(e);
    Graphics g = e.Graphics;

    // Rysowanie piłki
    g.FillEllipse(Brushes.Red, ballX, ballY, ballSize, ballSize);
}
```

- `Graphics` to kontekst graficzny, na którym wykonujemy całe rysowanie.

Grafika i animacja

- W konstruktorze formularza tworzymy i uruchamiamy `Timer`

```
public PongForm()
{
    InitializeComponent();
    this.DoubleBuffered = true; // zapobiega migotaniu

    timer = new Timer();
    timer.Interval = 20; // ms - częstotliwość odświeżania
    timer.Tick += Timer_Tick;
    timer.Start();
}
```

Grafika i animacja

- Metoda `TimerTick()` zapewni modyfikację położenia piłki i wyzwoli odświeżenie widoku.

```
private void Timer_Tick(object sender, EventArgs e)
{
    // Zmiana pozycji piłki
    ballX += dx;
    ballY += dy;

    // Odbicie od krawędzi formularza
    if (ballX <= 0 || ballX + ballSize >= this.ClientSize.Width)
        dx = -dx;

    if (ballY <= 0 || ballY + ballSize >= this.ClientSize.Height)
        dy = -dy;

    // Odśwież formularz
    this.Invalidate(); // wywoła OnPaint
}
```